

# Web Capabilities 程式碼研究室

來源：<https://codelabs.developers.google.com/codelabs/web-capabilities?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將使用幾種全新或只有在旗標才能使用的網頁 API。

---

# 目錄

1. [1. 介紹與設定](#)
2. [2. Badging API](#)
3. [3. Native File System API](#)
4. [4. Shape Detection API](#)
5. [5. Web Share Target API](#)
6. [6. Wake Lock API](#)
7. [7. Contact Picker API](#)
8. [8. Async Clipboard API](#)
9. [9. 恭喜完成！](#)

## 1. 介紹與設定

### 網頁功能

我們希望縮小網頁與原生應用程式之間的**功能差距**，讓開發人員輕鬆在開放網路上打造優質體驗。我們深信每位開發人員都應享有所需功能，打造優質的網路體驗，並致力於提升網路功能。

不過，有些功能 (例如檔案系統存取和閒置偵測) 僅適用於原生應用程式，不適用於網頁。缺少這些功能表示某些類型的應用程式無法在網路上提供，或實用性較低。

我們會以**公開透明的方式**設計及開發這些新功能，並採用現有的開放網路平台標準程序，同時在設計疊代期間，向開發人員和其他瀏覽器供應商徵求早期意見回饋，確保設計具有互通性。

歡迎查看我們目前正在開發的**所有功能**！

### 建構項目

在本程式碼研究室中，您將試用多項全新或僅在旗標後方提供的 Web API。因此，本程式碼研究室的重點是 API 本身，以及這些 API 帶來的用途，而不是建構特定的最終產品。

### 課程內容

本程式碼研究室會說明幾項尖端 API 的基本機制。請注意，這些機制尚未完全定案，我們非常重視您對開發人員流程的意見。

### 軟硬體需求

本程式碼研究室介紹的 API 都是最先進的技術，因此各 API 的需求有所不同。請務必詳閱各節開頭的相容性資訊。

### 如何進行程式碼研究室

這個程式碼研究室不一定需要依序完成。每個章節代表一個獨立的 API，您可以自由挑選最感興趣的內容。

---

步驟 2 · 約 3 分鐘

## 2. Badging API

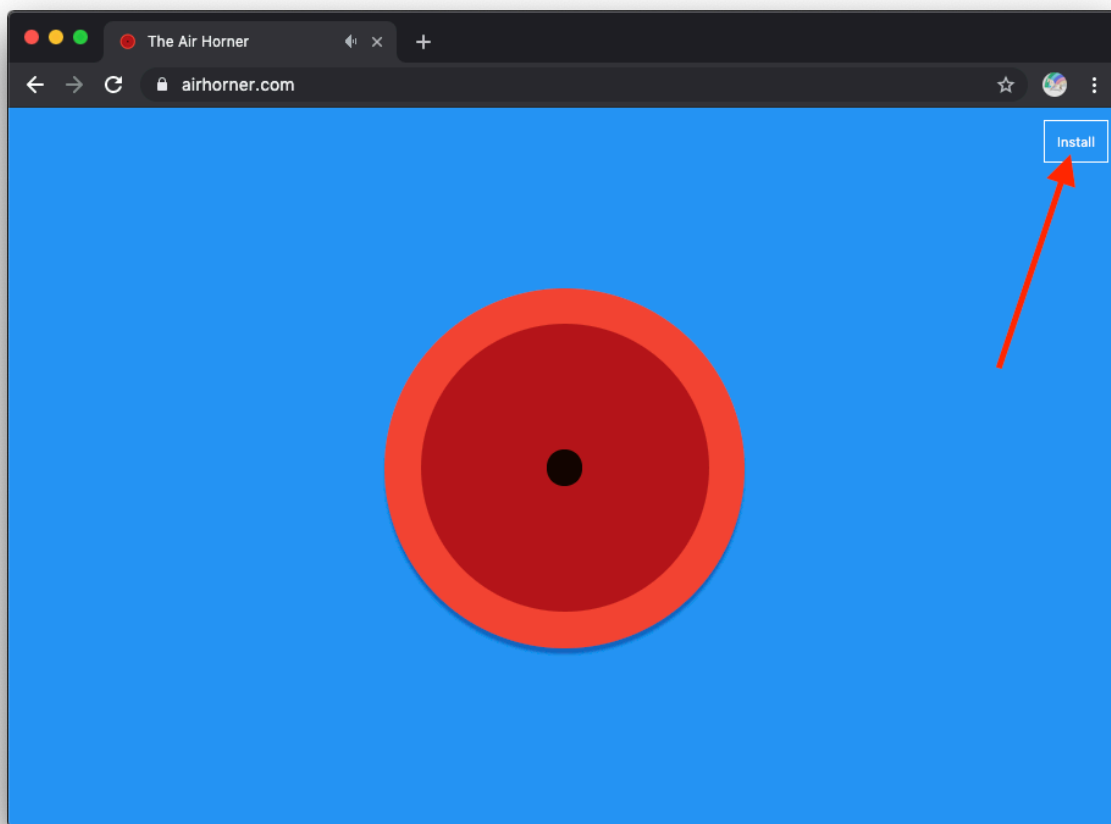
**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。本程式碼研究室以目前的設計為準，API 可能會在功能標準化前有所變動。如需最新詳情，請參閱 [Badging API 貼文](#)。

**相容性：**徽章 API 位於 `#enable-experimental-web-platform-features` 旗標後方，且為 Origin Trial。目前僅適用於 Windows 和 macOS，且僅適用於已安裝的 PWA。

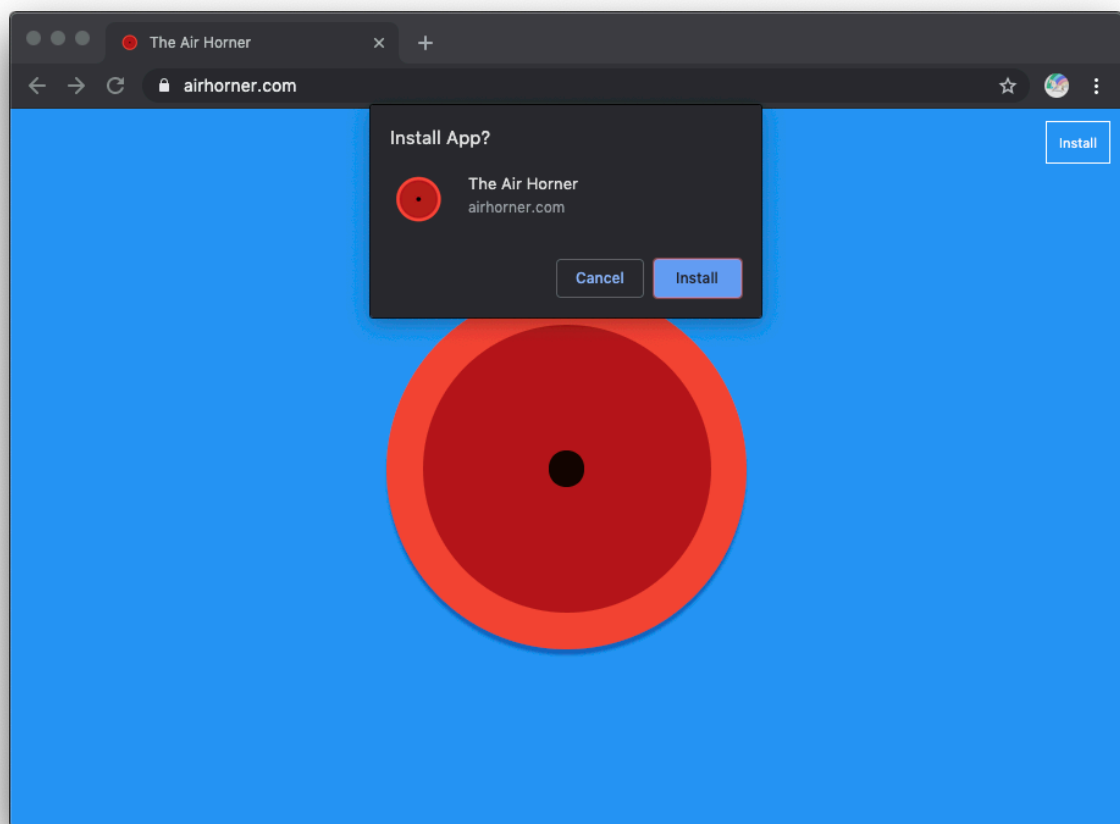
徽章 API 的目標是吸引使用者注意背景發生的事件。為了簡化本程式碼研究室的示範，我們將使用 API，讓使用者注意到前景發生的事件。然後將這些概念轉移到背景發生的事情。

### 安裝 Airhorner

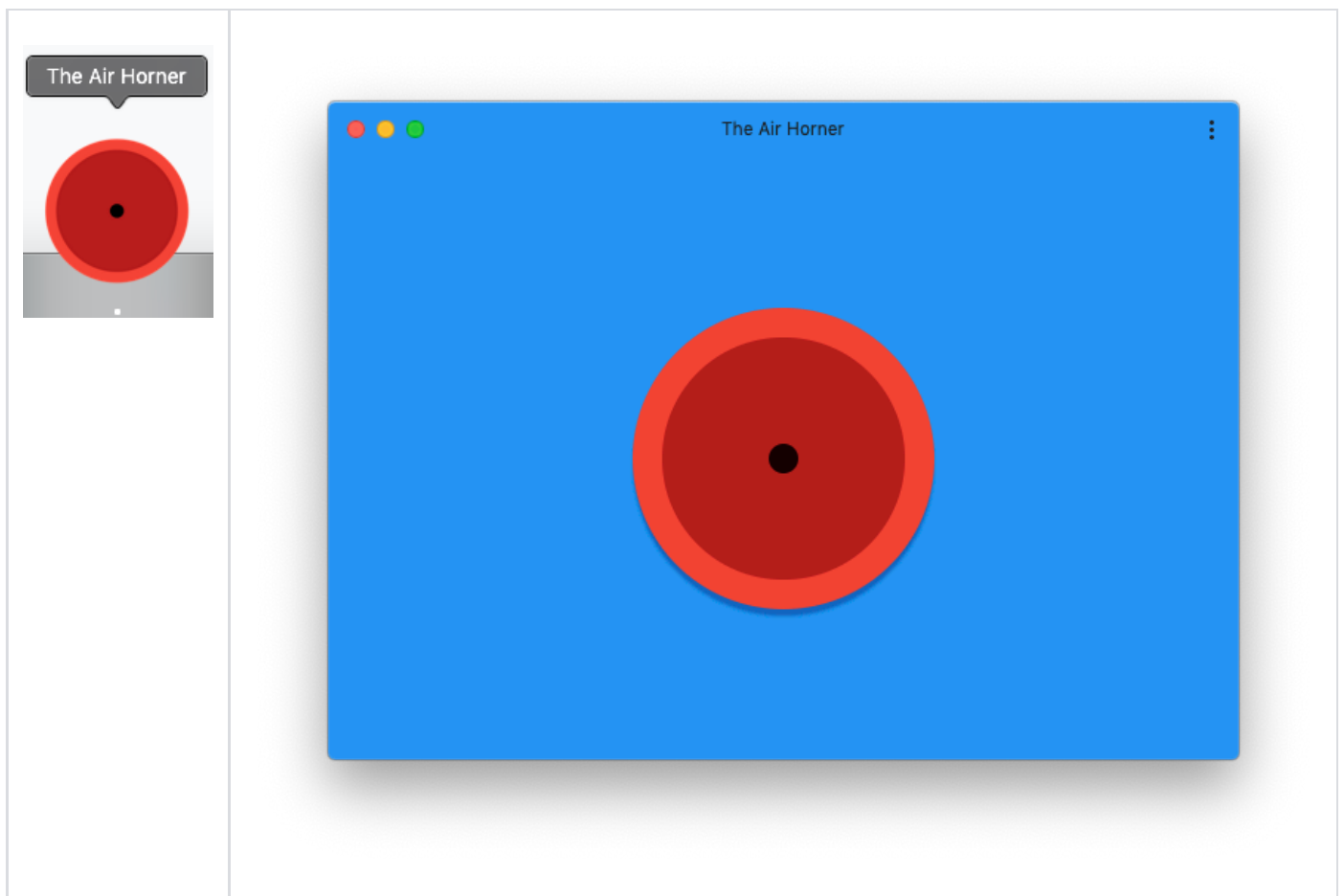
如要使用這項 API，您必須將 PWA 安裝到主畫面，因此第一步是安裝 PWA，例如惡名昭彰、舉世聞名的 [airhorner.com](#)。按下右上角的「安裝」按鈕，或使用三點圖示選單手動安裝。



系統會顯示確認提示，請按一下「安裝」。

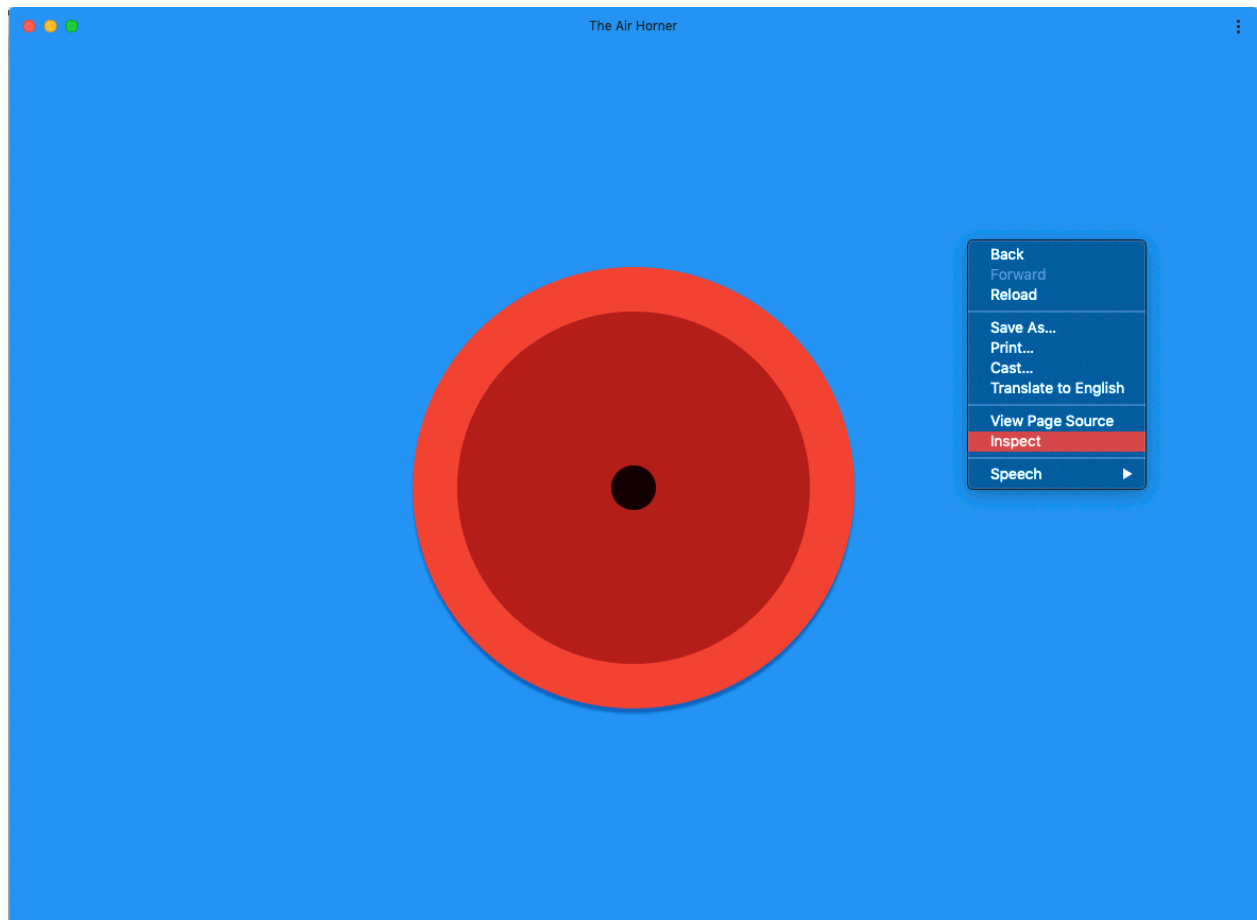


作業系統的 Dock 中現在會顯示新圖示。點選即可啟動 PWA。它將有自己的應用程式視窗，並在獨立模式下執行。



## 設定徽章

安裝 PWA 後，您需要一些數字資料 (徽章只能包含數字)，才能在徽章上顯示。在「The Air Horner」中，最容易計算的項目是嘆氣，也就是喇叭聲的次數。安裝 Airhorner 應用程式後，請嘗試吹響喇叭並檢查徽章。每按一次喇叭，系統就會增加一次計數。

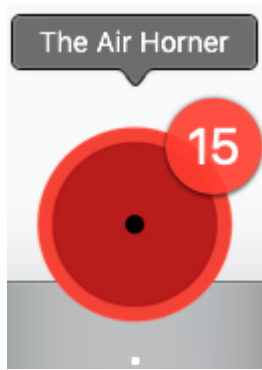


那麼，這項功能是如何運作的？程式碼基本上如下所示：

```
let hornCounter = 0;
const horn = document.querySelector('.horn');
horn.addEventListener('click', () => {
  navigator.setExperimentalAppBadge(++hornCounter);
});
```

**注意：**如果收到 `Uncaught ReferenceError`，請記得設定 `#enable-experimental-web-platform-features` 旗標。

按幾次氣笛，並檢查 PWA 的圖示：每次氣笛響起，圖示都會更新。就是這麼簡單。



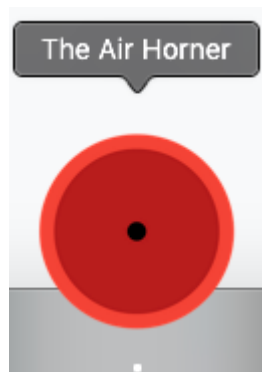
## 清除徽章

計數器會累加到 99，然後重新開始。你也可以手動重設。開啟開發人員工具的「控制台」分頁，貼上下方這行程式碼，然後按下 Enter 鍵。

```
navigator.setExperimentalAppBadge(0);
```

或者，您也可以明確清除徽章，如以下程式碼片段所示。PWA 圖示現在應該會再次顯示清楚的圖示，且沒有徽章。

```
navigator.clearExperimentalAppBadge();
```



## 意見回饋

你對這個 API 有什麼看法？請撥空填寫這份簡短的問卷調查，協助我們提供更優質的服務：

還有其他想法嗎？是否有缺少的功能？請填寫這份[問卷調查](#)，提供快速意見回饋。感謝您！

---

## 3. Native File System API

**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。本程式碼研究室以目前的設計為準，API 可能會在功能標準化前有所變動。如需最新詳細資料，請參閱[原生檔案系統 API 文章](#)。

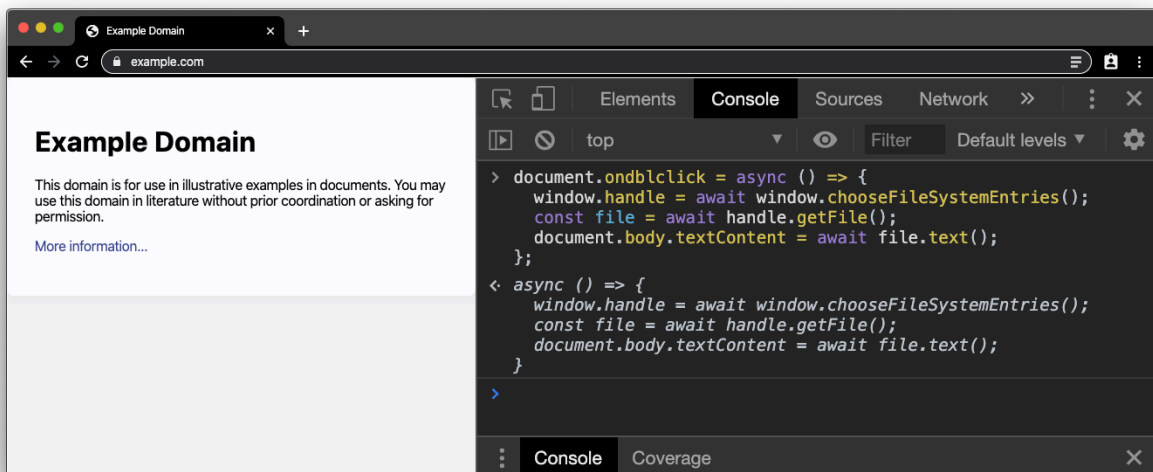
**相容性：**Native File System API 可透過 `#native-file-system-api` 旗標和 Origin Trial 取得。

開發人員可使用 Native File System API 建構強大的網頁應用程式，與使用者本機裝置上的檔案互動。使用者授予網頁應用程式存取權後，網頁應用程式就能透過這項 API 直接讀取或儲存使用者裝置上的檔案和資料夾。

### 讀取檔案

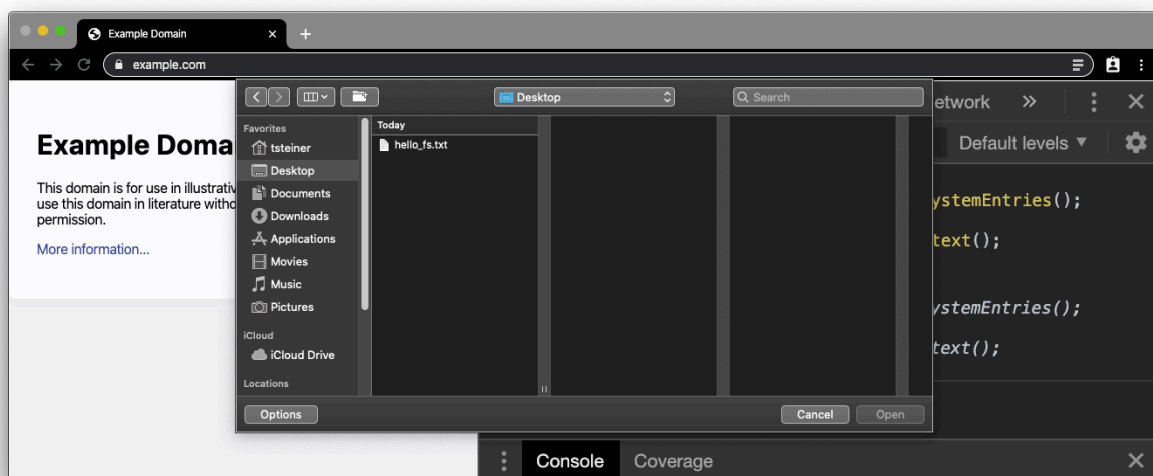
Native File System API 的「Hello, world」是讀取本機檔案並取得檔案內容。建立純文字 `.txt` 檔案，並輸入一些文字。接著，前往任何安全網站 (也就是透過 HTTPS 提供的網站)，例如 [example.com](#)，然後開啟開發人員工具控制台。將下列程式碼片段貼到控制台中。由於 Native File System API 需要使用者手勢，因此我們會在文件上附加按兩下處理常式。我們稍後會需要檔案控制代碼，因此請將其設為全域變數。

```
document.ondblclick = async () => {  
  window.handle = await window.chooseFileSystemEntries();  
  const file = await handle.getFile();  
  document.body.textContent = await file.text();  
};
```

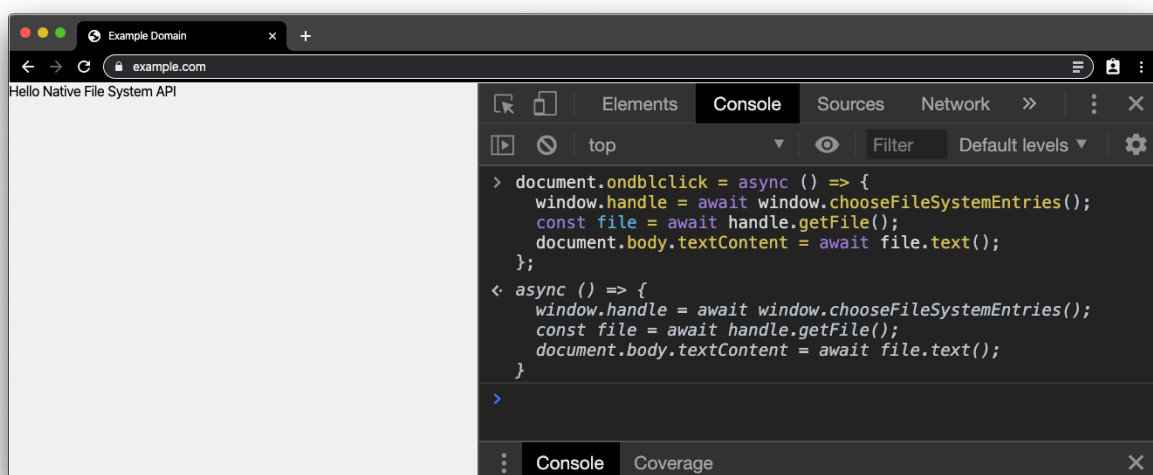


接著按兩下「example.com」[example.com](#) 頁面中的任何位置，系統就會顯示檔案選擇器。

**注意：**如果收到 `Uncaught ReferenceError`，請記得設定 `#native-file-system-api` 旗標。



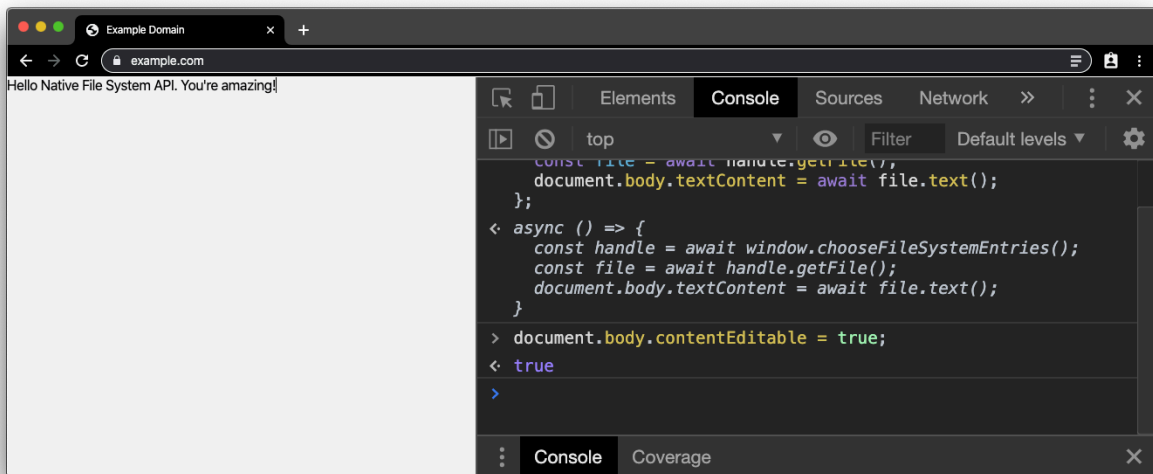
選取先前建立的 `.txt` 檔案。檔案內容隨即會取代 `example.com` 的實際 `body` 內容。



## 儲存檔案

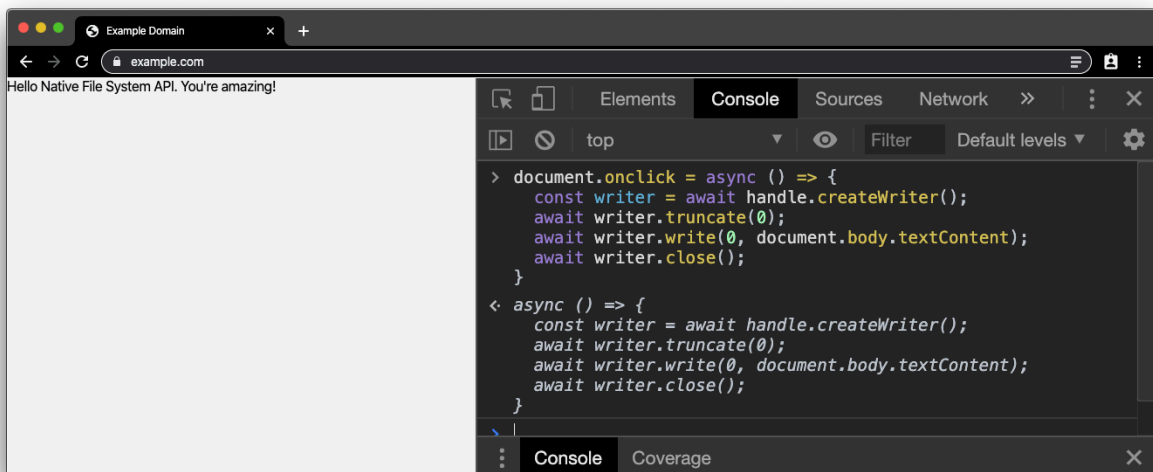
接著要進行一些變更。因此，請貼上下列程式碼片段，讓 `body` 可編輯。現在，您可以像使用文字編輯器一樣編輯文字。

```
document.body.contentEditable = true;
```

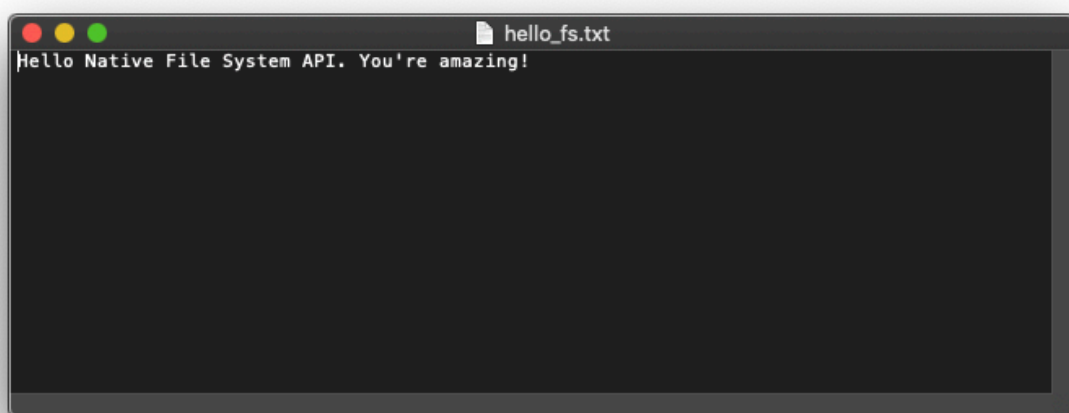
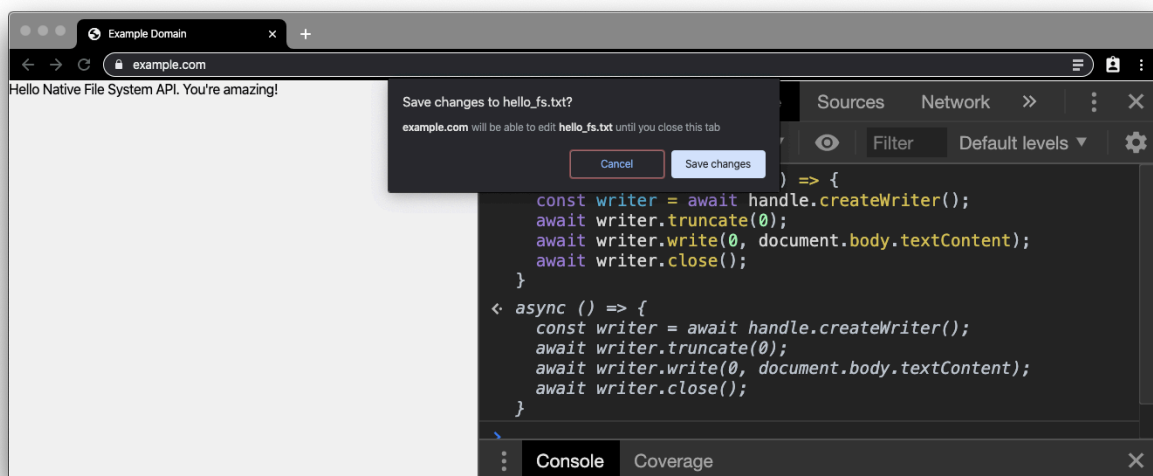


現在，我們要將這些變更寫回原始檔案。因此，我們需要檔案控制代碼的寫入器，方法是在控制台中貼上下方程式碼片段。我們再次需要使用者手勢，因此這次要等待點選主要文件。

```
document.onclick = async () => {
  const writer = await handle.createWriter();
  await writer.truncate(0);
  await writer.write(0, document.body.textContent);
  await writer.close();
};
```



現在按一下 (而非連按兩下) 文件時，系統會顯示權限提示。授予權限後，檔案內容會是您先前在 `body` 中編輯的內容。在其他編輯器中開啟檔案，或再次按兩下文件並重新開啟檔案，即可驗證變更。



恭喜！您剛才建立了全球最小的文字編輯器 [citation needed] 。

### 意見回饋

你對這個 API 有什麼看法？請撥空填寫這份簡短的問卷調查，協助我們提供更優質的服務：

還有其他想法嗎？是否有缺少的功能？請填寫這份[問卷調查](#)，提供快速意見回饋。感謝您！

## 4. Shape Detection API

**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。本程式碼研究室以目前的設計為準，API 可能會在功能標準化前有所變動。如需最新詳細資訊，請參閱 [Shape Detection API 文章](#)。

**相容性：**Shape Detection API 可透過 `#enable-experimental-web-platform-features` 標記使用。這項 API 的支援情況取決於硬體加速功能，而這類功能會因作業系統而異：

- `FaceDetector`：Android、macOS、Windows
- `BarcodeDetector`：Android、macOS
- `TextDetector`：Android、macOS (僅限邊界框)、Windows

Shape Detection API 可存取加速形狀偵測器 (例如人臉)，並適用於靜態圖片和/或即時圖片動態饋給。作業系統具有高效能且經過高度最佳化的特徵偵測器，例如 Android `FaceDetector`。Shape Detection API 會開放這些原生實作項目，並透過一組 JavaScript 介面公開。

目前支援的功能包括透過 `FaceDetector` 介面進行臉部偵測、透過 `BarcodeDetector` 介面進行條碼偵測，以及透過 `TextDetector` 介面進行文字偵測 (光學字元辨識)。

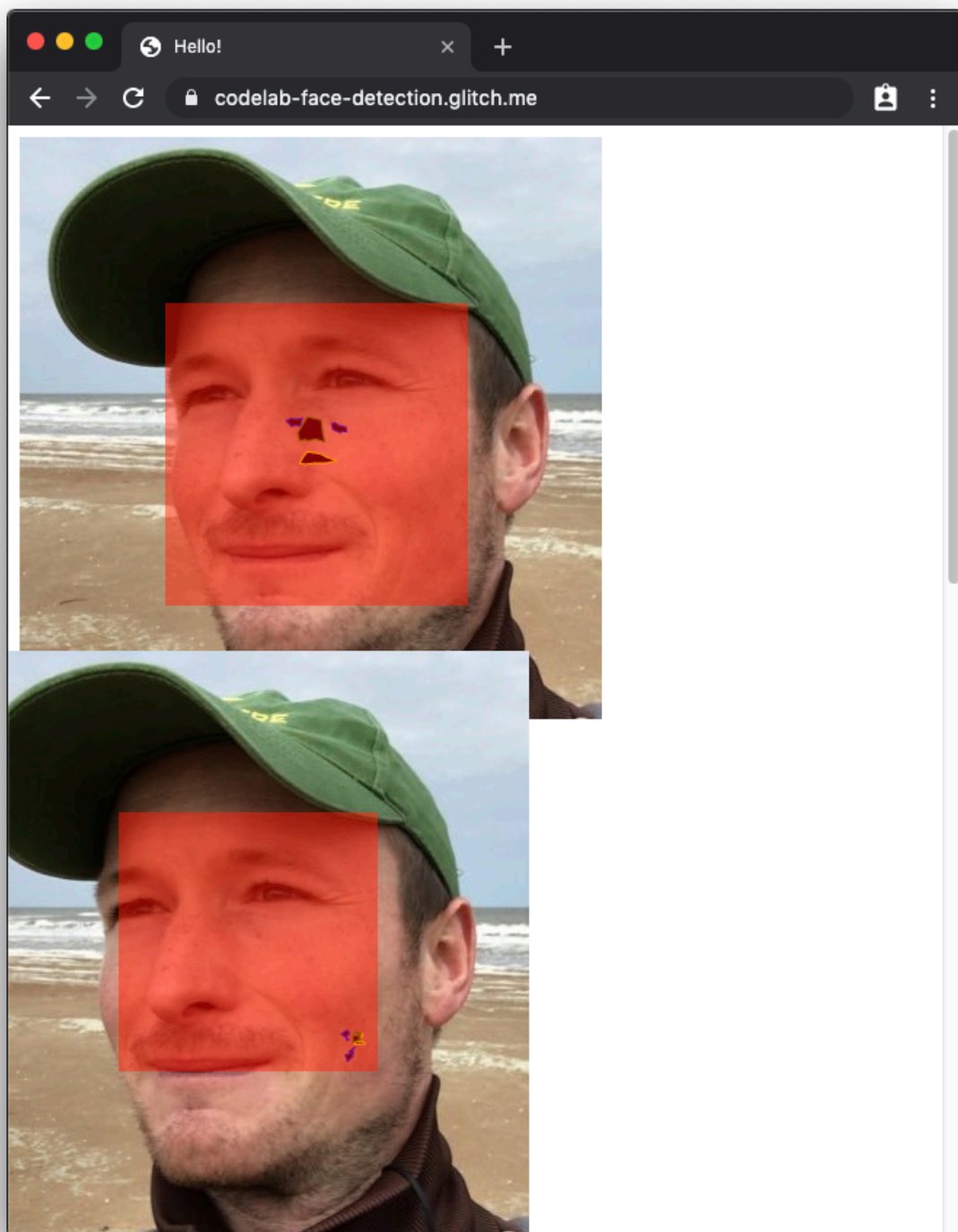
### 臉部偵測

Shape Detection API 的一項有趣功能是臉部偵測。如要測試這項功能，我們需要有臉部的網頁。[這個頁面](#)顯示作者的臉部，是不錯的開始。如下方螢幕截圖所示。在支援的瀏覽器上，系統會辨識臉部的邊界方塊和臉部地標。

您可能會發現，目前螢幕截圖中的臉部特徵點看起來不太正確。這是因為 [crbug.com/914348](https://crbug.com/914348)

┐\_(ツ)\_┐。

如要瞭解完成這項操作需要多少程式碼，請重新混音或編輯 [Glitch 專案](#)，尤其是 `script.js` 檔案。



如果想完全動態化，而不只是使用作者的臉部，請在私人分頁或訪客模式中，前往這個[Google 搜尋結果網頁](#)，其中包含許多臉部。現在，在該頁面上開啟 Chrome 開發人員工具，方法是在任意處按一下滑鼠右鍵，然後點選「檢查」。接著，在「控制台」分頁中貼上下列程式碼片段。程式碼會以半透明的紅色方塊標示偵測到的臉部。

請注意，一般來說，直接將程式碼複製並貼到正式版網站是個壞主意，Google 也會正確地警告您不要這麼做。不過，由於我們並未登入 (因此使用私密分頁或訪客模式)，因此在這裡這麼做沒問題。

注意：如果收到 `Uncaught ReferenceError`，請記得設定 `#enable-experimental-web-platform-features` 旗標。

```
document.querySelectorAll('img[alt]:not([alt=""])').forEach(async (img) => {
  try {
    const faces = await new FaceDetector().detect(img);
    faces.forEach(face => {
      const div = document.createElement('div');
      const box = face.boundingBox;
      const computedStyle = getComputedStyle(img);
      const [top, right, bottom, left] = [
        computedStyle.marginTop,
        computedStyle.marginRight,
        computedStyle.marginBottom,
        computedStyle.marginLeft
      ].map(m => parseInt(m, 10));
      const scaleX = img.width / img.naturalWidth;
      const scaleY = img.height / img.naturalHeight;
      div.style.backgroundColor = 'rgba(255, 0, 0, 0.5)';
      div.style.position = 'absolute';
      div.style.top = `${scaleY * box.top + top}px`;
      div.style.left = `${scaleX * box.left + left}px`;
      div.style.width = `${scaleX * box.width}px`;
      div.style.height = `${scaleY * box.height}px`;
      img.before(div);
    });
  } catch(e) {
    console.error(e);
  }
});
```

你會發現有一些 `DOMException` 訊息，而且並非所有圖片都會處理。這是因為「首要內容」圖片會內嵌為資料 URI，因此可以存取，而「非首要內容」圖片來自未設定支援 CORS 的其他網域。為了示範，我們不必擔心這個問題。

## 臉部特徵偵測

除了臉孔本身，macOS 也支援偵測臉部特徵點。如要測試臉部地標的偵測功能，請將下列程式碼片段貼到控制台中。提醒：由於 [crbug.com/914348](https://crbug.com/914348)，地標的排列順序並不完美，但您可以瞭解這項功能的發展方向和強大之處。

```

document.querySelectorAll('img[alt]:not([alt=""])').forEach(async (img) => {
  try {
    const faces = await new FaceDetector().detect(img);
    faces.forEach(face => {
      const div = document.createElement('div');
      const box = face.boundingBox;
      const computedStyle = getComputedStyle(img);
      const [top, right, bottom, left] = [
        computedStyle.marginTop,
        computedStyle.marginRight,
        computedStyle.marginBottom,
        computedStyle.marginLeft
      ].map(m => parseInt(m, 10));
      const scaleX = img.width / img.naturalWidth;
      const scaleY = img.height / img.naturalHeight;
      div.style.backgroundColor = 'rgba(255, 0, 0, 0.5)';
      div.style.position = 'absolute';
      div.style.top = `${scaleY * box.top + top}px`;
      div.style.left = `${scaleX * box.left + left}px`;
      div.style.width = `${scaleX * box.width}px`;
      div.style.height = `${scaleY * box.height}px`;
      img.before(div);

      const landmarkSVG = document.createElementNS('http://www.w3.org/2000/svg', 'svg');
      landmarkSVG.style.position = 'absolute';
      landmarkSVG.classList.add('landmarks');
      landmarkSVG.setAttribute('viewBox', `0 0 ${img.width} ${img.height}`);
      landmarkSVG.style.width = `${img.width}px`;
      landmarkSVG.style.height = `${img.height}px`;
      face.landmarks.map((landmark) => {
        landmarkSVG.innerHTML += `<polygon class="landmark-${landmark.type}" points="${
          landmark.locations.map((point) => {
            return `${scaleX * point.x},${scaleY * point.y}`;
          }).join(' ')
        }" /></svg>`;
      });
      div.before(landmarkSVG);
    });
  } catch(e) {
    console.error(e);
  }
});

```

## 條碼偵測

Shape Detection API 的第二項功能是條碼偵測。與先前類似，我們需要含有條碼的頁面，例如[這個頁面](#)。在瀏覽器中開啟後，您會看到解讀出的各種 QR code。混用或編輯 [Glitch 專案](#)，尤其是 [script.js 檔案](#)，即可瞭解做法。



如要使用更動態的圖片，可以再次使用 Google 圖片搜尋。這次請在瀏覽器中，以私密分頁或訪客模式前往這個 [Google 搜尋結果網頁](#)。現在請將下列程式碼片段貼到 Chrome 開發人員工具的「控制台」分頁中。過一會兒，系統就會在辨識出的條碼上標註原始值和條碼類型。

```
document.querySelectorAll('img[alt]:not([alt=""])').forEach(async (img) => {
  try {
    const barcodes = await new BarcodeDetector().detect(img);
    barcodes.forEach(barcode => {
      const div = document.createElement('div');
      const box = barcode.boundingBox;
      const computedStyle = getComputedStyle(img);
      const [top, right, bottom, left] = [
        computedStyle.marginTop,
        computedStyle.marginRight,
        computedStyle.marginBottom,
        computedStyle.marginLeft
      ].map(m => parseInt(m, 10));
      const scaleX = img.width / img.naturalWidth;
      const scaleY = img.height / img.naturalHeight;
      div.style.backgroundColor = 'rgba(255, 255, 255, 0.75)';
      div.style.position = 'absolute';
      div.style.top = `${scaleY * box.top + top}px`;
      div.style.left = `${scaleX * box.left - left}px`;
      div.style.width = `${scaleX * box.width}px`;
      div.style.height = `${scaleY * box.height}px`;
      div.style.color = 'black';
      div.style.fontSize = '14px';
      div.textContent = `${barcode.rawValue}`;
      img.before(div);
    });
  } catch(e) {
    console.error(e);
  }
});
```

## 文字偵測

Shape Detection API 的最後一項功能是文字偵測。您應該已經瞭解：我們需要含有文字的圖片頁面，例如[這個頁面](#)，其中包含 Google 圖書掃描結果。在支援的瀏覽器中，系統會辨識文字，並在文字段落周圍繪製定界框。混用或編輯 [Glitch 專案](#)，尤其是 [script.js 檔案](#)，即可瞭解做法。

THE  
MAGAZINE OF NATURAL HISTORY, HISTORY

AND  
JOURNAL  
OF  
ZOOLOGY, MINERALOGY, GEOLOGY,  
BOTANY, AND METEOROLOGY,



CONDUCTED  
By J. C. LOUDON, F.L.S. G.S. &c.  
MEMBER OF THE ZOOLOGICAL  
SOCIETY OF LONDON, AND OF  
VARIOUS NATURAL HISTORY  
SOCIETIES ON THE CONTINENT.

LONDON:  
PRINTED FOR LONGMAN, REES, ORME,  
BROWN, AND GREEN, PATERNOSTER-ROW,  
1829.

如要動態測試這項功能，請在無痕分頁或訪客模式中前往這個[搜尋結果網頁](#)。現在請將下列程式碼片段貼到 Chrome 開發人員工具的「控制台」分頁中。稍待片刻，系統就會辨識部分文字。

```
document.querySelectorAll('img[alt]:not([alt=""])').forEach(async (img) => {
  try {
    const texts = await new TextDetector().detect(img);
    texts.forEach(text => {
      const div = document.createElement('div');
      const box = text.boundingBox;
      const computedStyle = getComputedStyle(img);
      const [top, right, bottom, left] = [
        computedStyle.marginTop,
        computedStyle.marginRight,
        computedStyle.marginBottom,
        computedStyle.marginLeft
      ].map(m => parseInt(m, 10));
      const scaleX = img.width / img.naturalWidth;
      const scaleY = img.height / img.naturalHeight;
      div.style.backgroundColor = 'rgba(255, 255, 255, 0.75)';
      div.style.position = 'absolute';
      div.style.top = `${scaleY * box.top + top}px`;
      div.style.left = `${scaleX * box.left - left}px`;
      div.style.width = `${scaleX * box.width}px`;
      div.style.height = `${scaleY * box.height}px`;
      div.style.color = 'black';
      div.style.fontSize = '14px';
      div.innerHTML = text.rawValue;
      img.before(div);
    });
  } catch(e) {
    console.error(e);
  }
});
```

## 意見回饋

你對這個 API 有什麼看法？請撥空填寫這份簡短的問卷調查，協助我們提供更優質的服務：

還有其他想法嗎？是否有缺少的功能？請填寫這份[問卷調查](#)，提供快速意見回饋。感謝您！

---

## 5. Web Share Target API

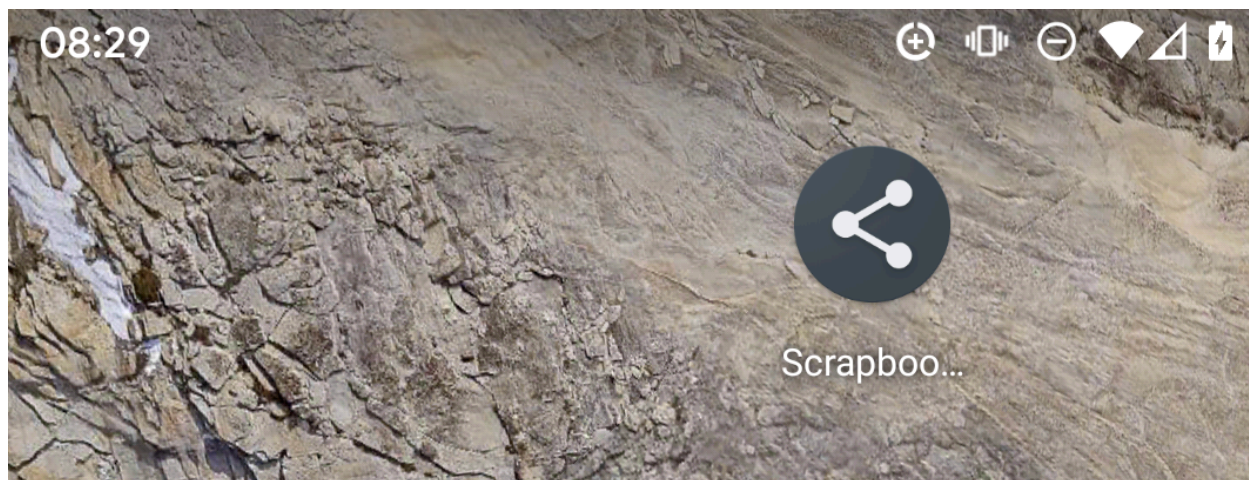
**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。這個 API 現已推出 🚧，本程式碼研究室以目前的穩定版設計為基礎。如需最新詳細資料，請參閱 [Web Share Target API 貼文](#)。

**相容性：**Web Share Target API 適用於 Android、Windows、Linux 和 ChromeOS。

Web Share Target API 可讓已安裝的網頁應用程式向底層作業系統註冊為分享目標，以便接收來自 Web Share API 或系統事件 (例如作業系統層級的分享按鈕) 的分享內容。

### 安裝 PWA 以分享至

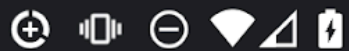
首先，您需要可分享內容的 PWA。這次 Airhorner (幸運地) 不會完成這項工作，但 [Web Share Target 試用版應用程式](#) 會支援這項功能。將應用程式安裝到裝置主畫面。



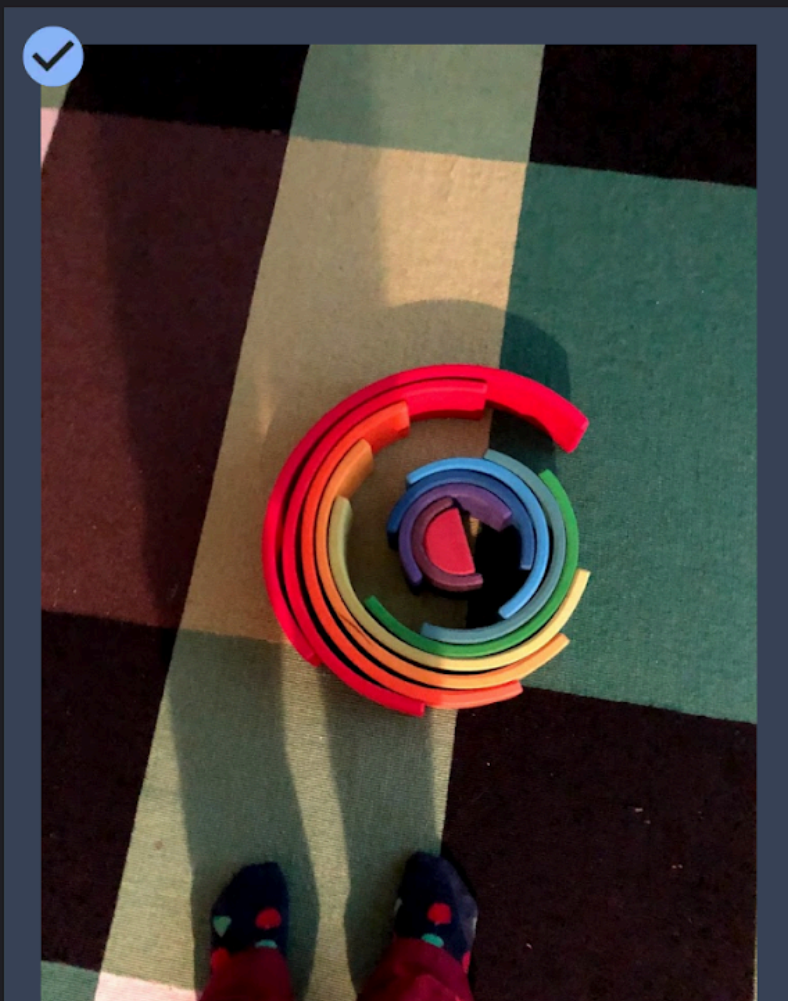
### 將內容分享至 PWA

接著，您需要分享內容，例如 Google 相簿中的相片。使用「分享」按鈕，然後選取 Scrapbook PWA 做為分享目標。

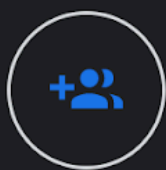
08:27



1

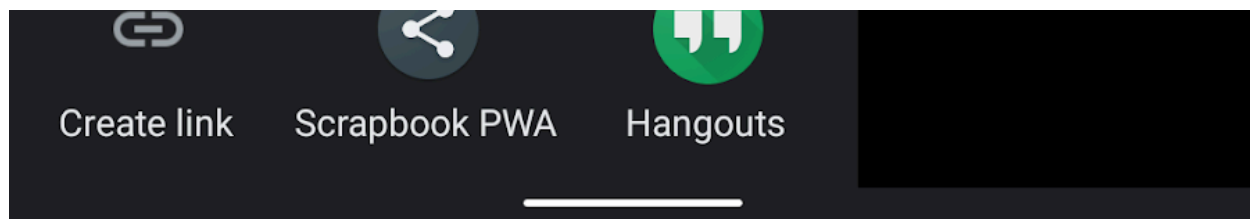


SEND IN GOOGLE PHOTOS



New group

SHARE TO APPS



輕觸應用程式圖示後，系統會直接開啟剪貼簿 PWA，並顯示相片。

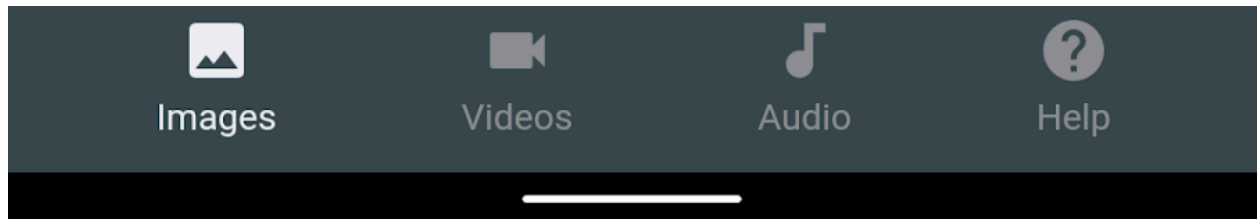
08:27



VIEW

SHARE

DELETE



那麼，這項功能是如何運作的？如要瞭解詳情，請參閱 Scrapbook PWA 的[網頁應用程式資訊清單](#)。如要讓 Web Share Target API 運作，請在資訊清單的 `"share_target"` 屬性中進行設定，該屬性的 `"action"` 欄位會指向一個網址，該網址會以 `"params"` 中列出的參數進行裝飾。

分享端會相應填入這個網址範本 (透過分享動作輔助，或由開發人員[使用 Web Share API](#) 以程式輔助控制)，接收端隨後就能擷取參數並加以運用，例如顯示參數。

```
{
  "action": "/_share-target",
  "enctype": "multipart/form-data",
  "method": "POST",
  "params": {
    "files": [{
      "name": "media",
      "accept": ["audio/*", "image/*", "video/*"]
    }]
  }
}
```

## 意見回饋

你對這個 API 有什麼看法？請撥空填寫這份簡短的問卷調查，協助我們提供更優質的服務：

還有其他想法嗎？是否有缺少的功能？請填寫這份[問卷調查](#)，提供快速意見回饋。感謝您！

---

## 6. Wake Lock API

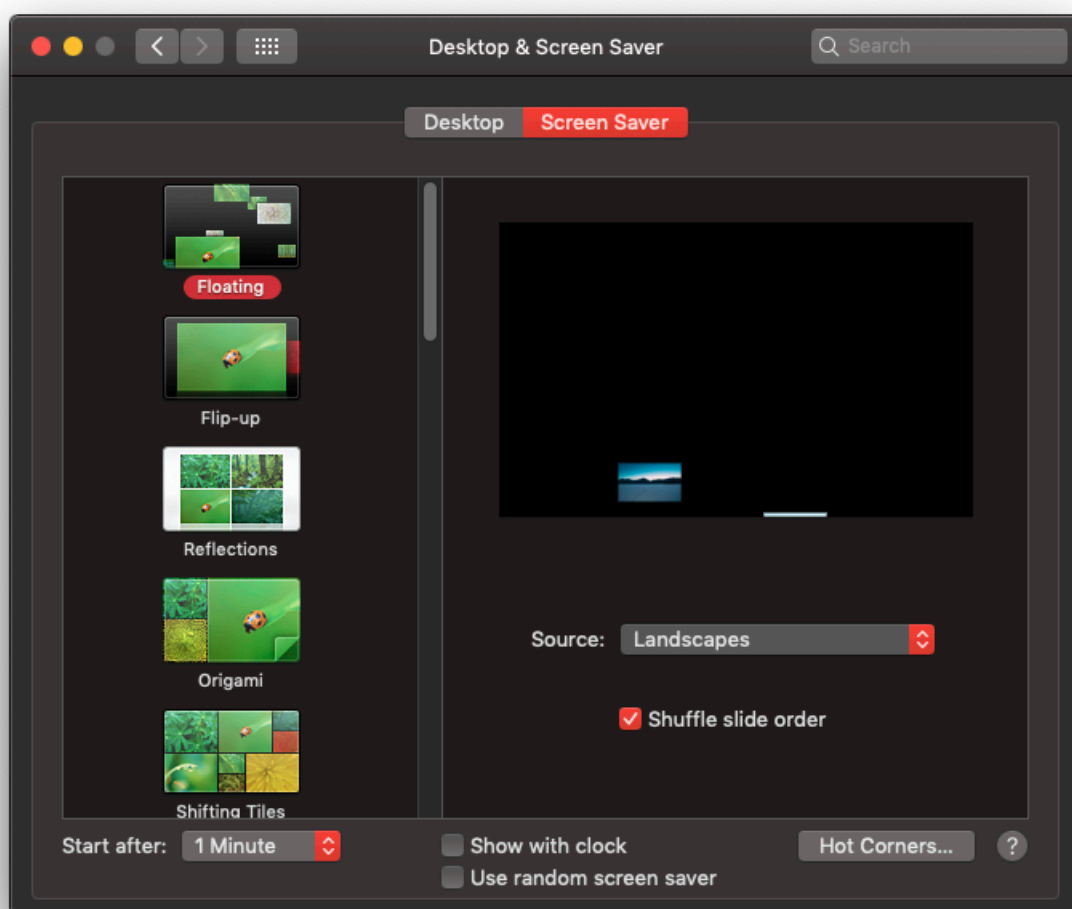
**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。本程式碼研究室以目前的設計為準，API 可能會在功能標準化前有所變動。如需最新詳細資料，請參閱「[Wake Lock API](#)」一文。

**相容性：**在 Android、Windows、Linux 和 macOS 上，Wake Lock API 位於 `#enable-experimental-web-platform-features` 旗標後方。

為避免耗盡電力，大多數裝置在閒置時會快速進入休眠狀態。雖然大部分時間都沒問題，但有些應用程式需要讓螢幕或裝置保持喚醒狀態，才能完成工作。Wake Lock API 可防止裝置變暗和鎖定螢幕，或防止裝置進入休眠狀態。這項功能可提供全新體驗，而這類體驗過去只能透過原生應用程式提供。

### 設定螢幕保護程式

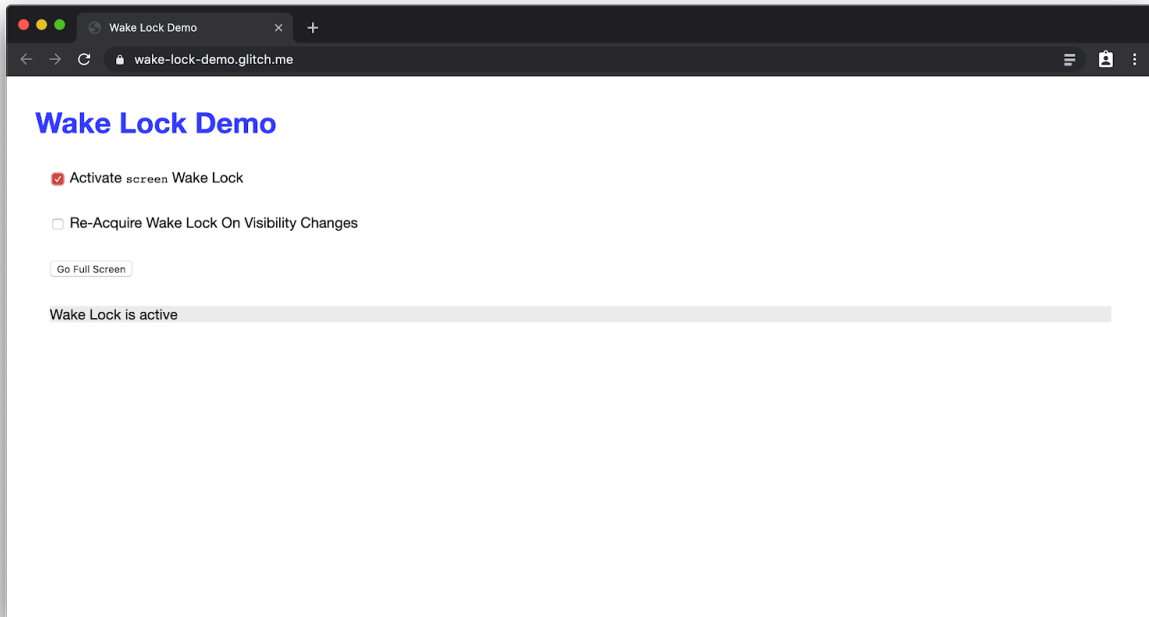
如要測試 Wake Lock API，請先確保裝置會進入休眠狀態。因此，請在作業系統的偏好設定窗格中啟動所選的螢幕保護程式，並確認螢幕保護程式會在 1 分鐘後啟動。請讓裝置閒置一段時間（沒錯，我知道這很痛苦），確認裝置是否正常運作。以下螢幕截圖顯示的是 macOS，但你當然可以在 Android 裝置或任何支援的電腦平台嘗試這項功能。



# 設定螢幕喚醒鎖定

現在您已瞭解螢幕保護程式的運作方式，接下來請使用 `"screen"` 類型的 Wake Lock，防止螢幕保護程式執行工作。前往 [Wake Lock 試用版應用程式](#)，然後按一下「Activate」（啟用）

`screen` 勾選「Wake Lock」核取方塊。



從那一刻起，Wake Lock 就會啟用。只要耐心等待一分鐘，不要觸碰裝置，就會發現螢幕保護程式確實沒有啟動。

那麼，這項功能如何運作？如要瞭解詳情，請前往 Wake Lock 試用版應用程式的 [Glitch 專案](#)，並查看 [script.js](#)。程式碼的要點如下列程式碼片段所示。開啟新分頁 (或使用任何已開啟的分頁)，然後將下列程式碼貼到 Chrome 開發人員工具控制台中。按一下視窗後，您應該會看到 Wake Lock 狀態正好維持 10 秒 (請參閱控制台記錄)，且螢幕保護程式不會啟動。

**注意：**如果收到 `Uncaught ReferenceError`，請記得設定 `#enable-experimental-web-platform-features` 旗標。

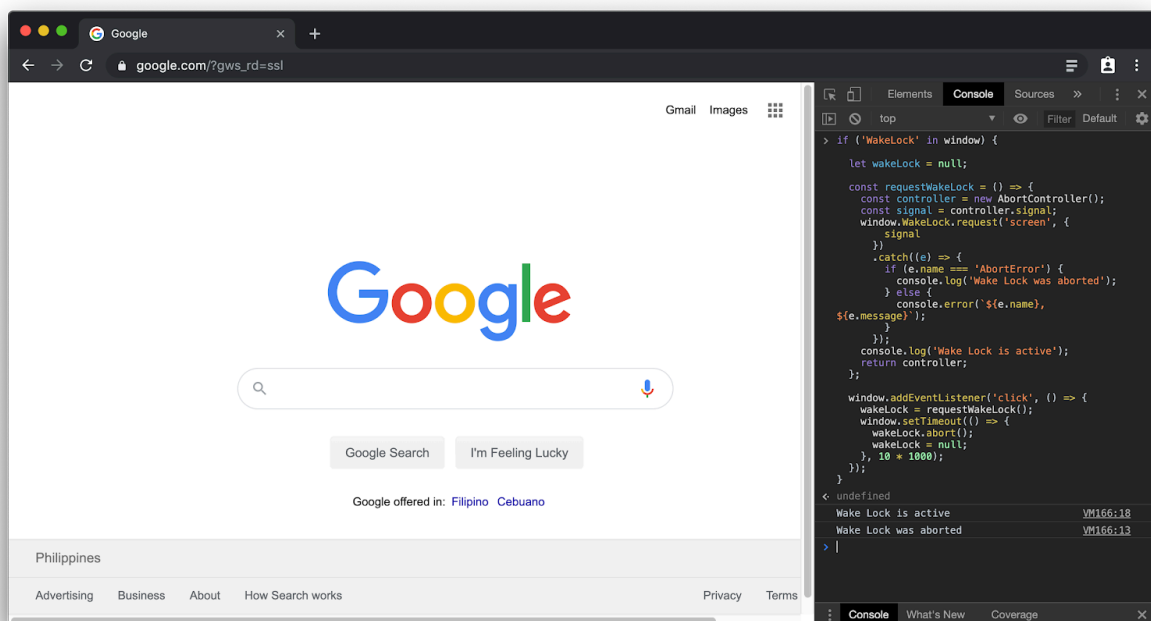
```

if ('wakeLock' in navigator && 'request' in navigator.wakeLock) {
  let wakeLock = null;

  const requestWakeLock = async () => {
    try {
      wakeLock = await navigator.wakeLock.request('screen');
      wakeLock.addEventListener('release', () => {
        console.log('Wake Lock was released');
      });
      console.log('Wake Lock is active');
    } catch (e) {
      console.error(`${e.name}, ${e.message}`);
    }
  };

  requestWakeLock();
  window.setTimeout(() => {
    wakeLock.release();
  }, 10 * 1000);
}

```



## 意見回饋

你對這個 API 有什麼看法？請撥空填寫這份簡短的問卷調查，協助我們提供更優質的服務：

還有其他想法嗎？是否有缺少的功能？請填寫這份問卷調查，提供快速意見回饋。感謝您！

## 7. Contact Picker API

**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。本程式碼研究室以目前的設計為準，API 可能會在功能標準化前有所變動。如需最新詳細資料，請參閱「[聯絡人選擇工具 API](#)」一文。

**相容性：**Android 上的 Wake Lock API 位於 `#enable-experimental-web-platform-features` 旗標後方。

我們非常期待聯絡人選擇工具 API 的推出。這項功能可讓網頁應用程式存取裝置原生聯絡人管理工具中的聯絡人，因此網頁應用程式可以存取聯絡人的姓名、電子郵件地址和電話號碼。你可以指定要匯出單一或多個聯絡人，以及要匯出所有欄位，還是只匯出名稱、電子郵件地址和電話號碼等部分欄位。

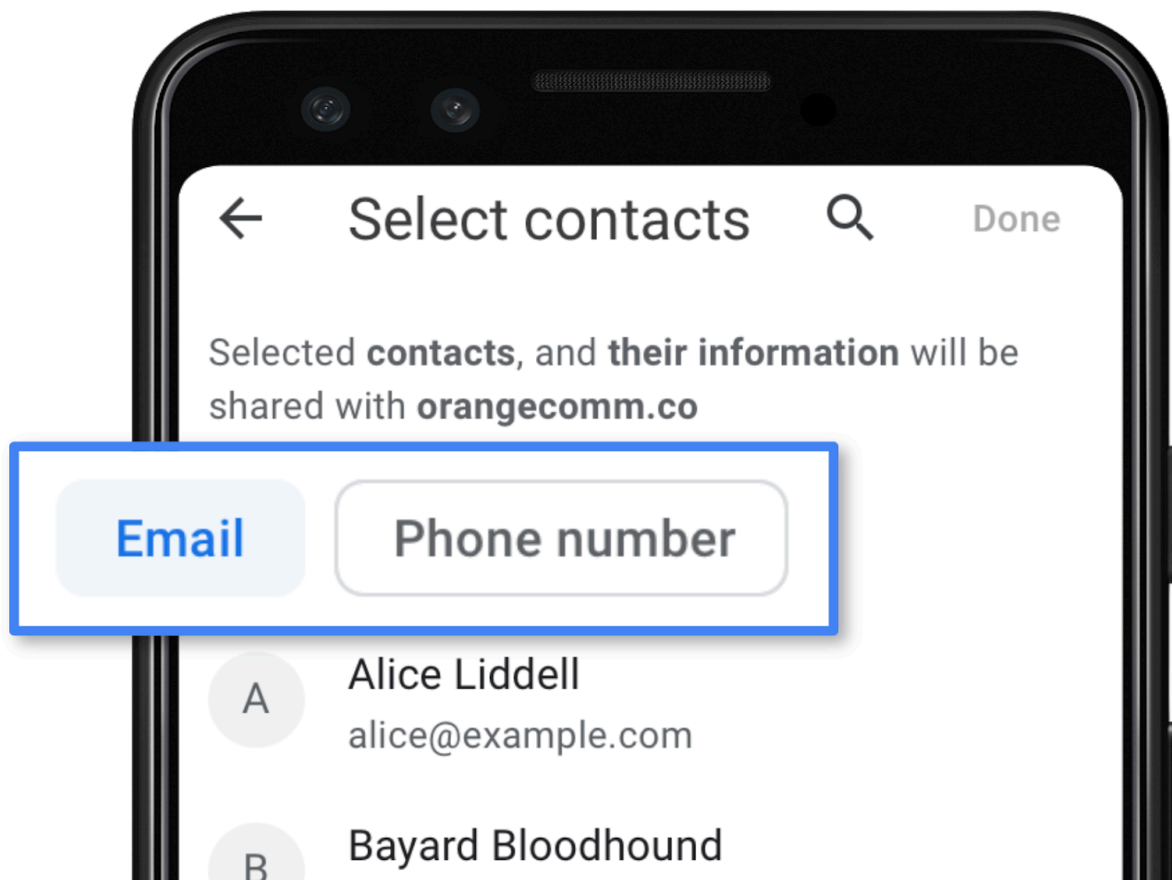
### 隱私權考量

開啟挑選器後，即可選取要分享的聯絡人。請注意，系統不會提供「全選」選項，這是刻意設計，因為我們希望使用者能審慎決定要分享的內容。同樣地，存取權並非持續授予，而是單次決定。

### 存取聯絡人

存取聯絡人非常簡單，開啟挑選器前，您可以指定要哪些欄位 (選項為 `name`、`email` 和 `telephone`)，以及要存取多個還是只有一個聯絡人。您可以在 Android 裝置上開啟[示範應用程式](#)，測試這項 API。[原始碼](#)的相關部分基本上是下列程式碼片段：

```
getContactsButton.addEventListener('click', async () => {
  const contacts = await navigator.contacts.select(
    ['name', 'email'],
    {multiple: true});
  if (!contacts.length) {
    // No contacts were selected, or picker couldn't be opened.
    return;
  }
  console.log(contacts);
});
```



## 8. Async Clipboard API

**注意：**我們正在開發這項 API，這是新版「功能」專案的一部分。這個 API 現已推出 🚧，本程式碼研究室以目前的穩定版設計為基礎。如需最新詳細資料，請參閱「[非同步剪貼簿 API](#)」一文。

### 複製及貼上文字

到目前為止，還無法以程式輔助方式將圖片複製及貼到系統剪貼簿。我們最近在 Async Clipboard API 中新增了圖片支援功能，

現在可以複製及貼上圖片。新功能是您也可以將圖片寫入剪貼簿。非同步剪貼簿 API 支援複製及貼上文字已有一段時間。您可以呼叫 `navigator.clipboard.writeText()` 將文字複製到剪貼簿，然後呼叫 `navigator.clipboard.readText()` 貼上該文字。

### 複製及貼上圖片

現在也可以將圖片寫入剪貼簿。如要讓這項功能正常運作，您需要將圖片資料做為 Blob，然後傳遞至剪貼簿項目建構函式。最後，您可以呼叫 `navigator.clipboard.write()` 複製這個剪貼簿項目。

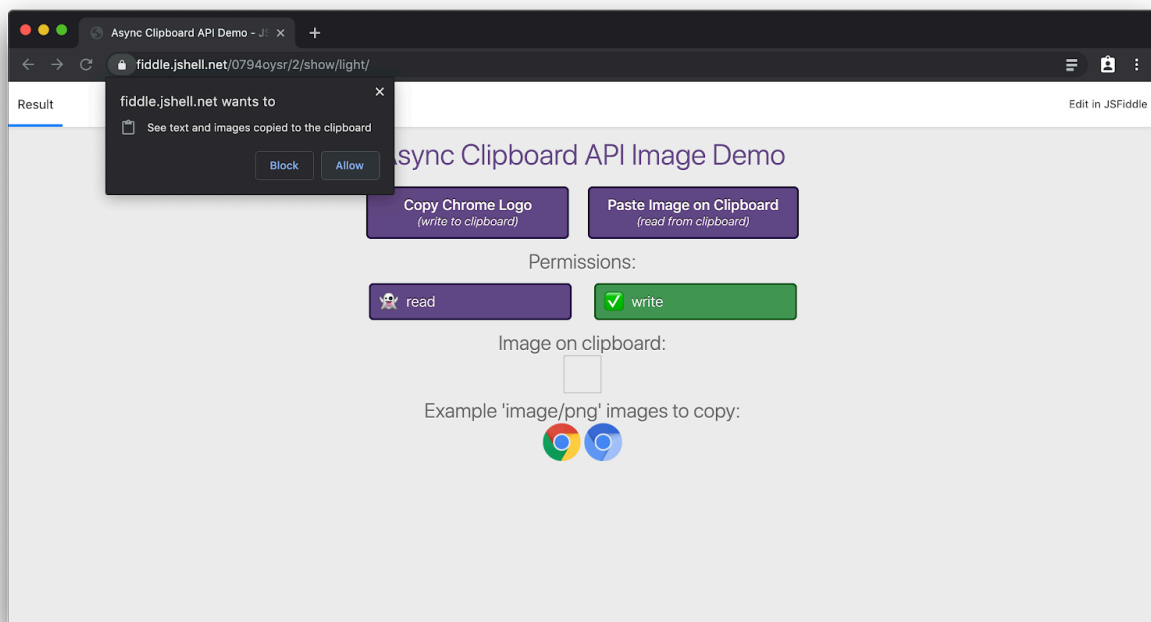
```
// Copy: Writing image to the clipboard
try {
  const imgURL = 'https://developers.google.com/web/updates/images/generic/file.png';
  const data = await fetch(imgURL);
  const blob = await data.blob();
  await navigator.clipboard.write([
    new ClipboardItem(Object.defineProperty({}, blob.type, {
      value: blob,
      enumerable: true
    })))
  ];
  console.log('Image copied.');
```

```
} catch(e) {
  console.error(e, e.message);
}
```

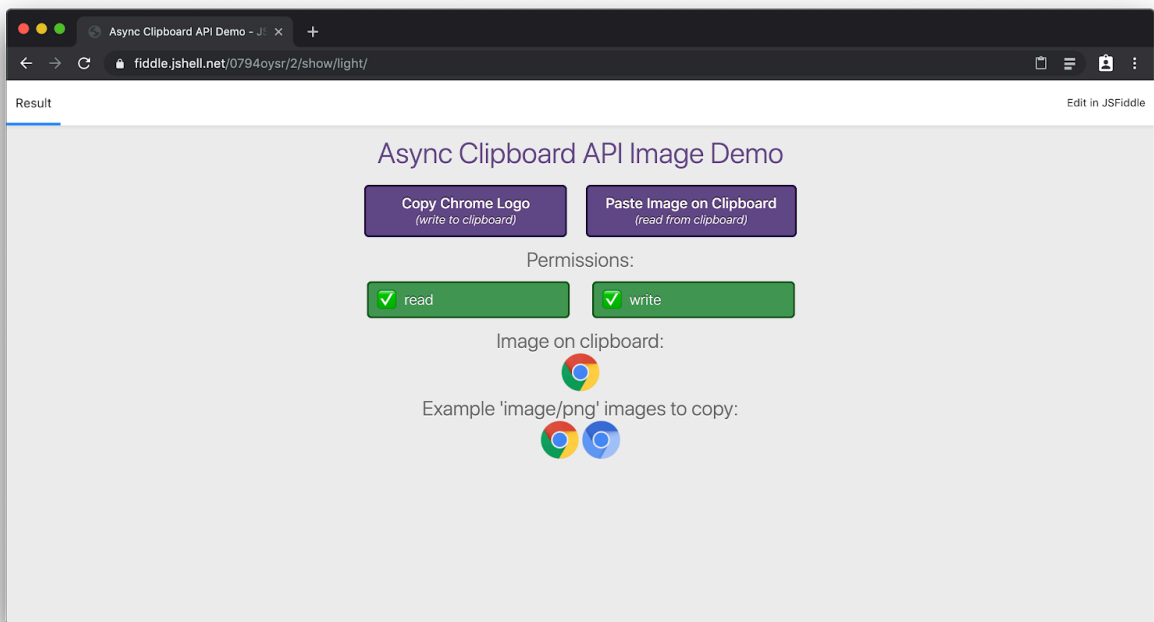
從剪貼簿貼回圖片的程序看似複雜，但其實只是從剪貼簿項目取回 Blob。由於可能有多個，您需要逐一檢查，直到找到感興趣的項目為止。基於安全考量，目前僅支援 PNG 圖片，但日後可能會支援更多圖片格式。

```
async function getClipboardContents() {
  try {
    const clipboardItems = await navigator.clipboard.read();
    for (const clipboardItem of clipboardItems) {
      try {
        for (const type of clipboardItem.types) {
          const blob = await clipboardItem.getType(type);
          console.log(URL.createObjectURL(blob));
        }
      } catch (e) {
        console.error(e, e.message);
      }
    }
  } catch (e) {
    console.error(e, e.message);
  }
}
```

如要查看這個 API 的實際應用情形，請參閱[試用版應用程式](#)。上文已嵌入相關[原始碼](#)片段。複製圖片到剪貼簿時不需要權限，但貼上剪貼簿內容時需要授予存取權。



授予存取權後，您就可以從剪貼簿讀取圖片，並貼到應用程式中：



步驟 9 · 約 0 分鐘

## 9. 恭喜完成！

恭喜，您已經順利完成本程式碼研究室課程！再次提醒，大多數 API 仍在變動，且積極開發中。因此，團隊非常感謝您的[意見](#)，因為只有與您這類使用者互動，才能協助我們正確使用這些 API。

如果不想使用最新功能，現在可以將 `#enable-experimental-web-platform-features` 旗標設回 `Disabled`。

此外，建議您經常查看[功能到達網頁](#)。我們會持續更新這份文件，並提供我們所開發 API 的所有深入文章指標。繼續加油！

Tom 和整個功能團隊 🍌

---